

Structured Query Language - SQL

Parte 7

QXD0011 - Fundamentos de Bancos de Dados

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro

victorpinheiro@ufc.br



Agenda

- TUFCCitter
- Esquema do banco
- Questões
- Respostas

TUFCitter

- O banco TUFCitter representa um sistema de microblog onde usuários podem criar posts, curtir publicações, seguir outros usuários e trocar mensagens privadas. O banco registra informações pessoais (usuário, endereço, telefone, perfil), posts, curtidas, relações de seguidores e mensagens trocadas. O modelo permite consultas complexas sobre as interações entre os usuários da plataforma.

A seguir o esquema do banco

- Usuario (id_usuario, login, senha, data_cadastro, email)
- EnderecoUsuario (id_usuario, rua, numero, cidade, estado, cep)
- UsuarioTelefone (id_usuario, telefone)
- Perfil (id_usuario, biografia, foto_url)
- Post (id_post, id_usuario, texto, data_hora)
- Seguir (id_seguidor, id_seguido, data_hora)
- Curtida (id_usuario, id_post, data_hora)
- MensagemPrivada (id_msg, id_remetente, id_destinatario, conteudo, data_hora)

01. Total de Curtidas Recebidas por Usuário

- Técnica: JOIN, GROUP BY, HAVING, COUNT, ORDER BY
- Considere todos os usuários cadastrados no sistema. Para cada usuário, informe: O login (nome de usuário). O total de curtidas recebidas em todos os seus posts. O número total de posts que ele publicou. Exiba apenas os usuários que receberam mais de 3 curtidas no total, considerando a soma das curtidas de todos os seus posts. Apresente a lista ordenada, do usuário com maior número de curtidas para o menor.

Resposta

```
SELECT
    u.login,
    COUNT(c.id_post) AS total_curtidas,
    COUNT(DISTINCT p.id_post) AS total_posts
FROM Usuario u
LEFT JOIN Post p
    ON p.id_usuario = u.id_usuario
LEFT JOIN Curtida c
    ON c.id_post = p.id_post
GROUP BY u.id_usuario, u.login
HAVING COUNT(c.id_post) > 3
ORDER BY total_curtidas DESC;
```

Como funciona

- FROM Usuario
 - Começa com todos os usuários.
- LEFT JOIN Post
 - Liga cada usuário aos seus posts (usando id_usuario).
 - Usuário sem post fica com p.id_post = NULL.
- LEFT JOIN Curtida
 - Liga cada post às suas curtidas (usando id_post).
 - Post sem curtida fica com c.id_post = NULL.
- GROUP BY u.id_usuario, u.login
 - Agrupa por usuário, para poder somar/contar sobre os posts e curtidas dele.
- COUNT(c.id_post)
 - Conta quantas curtidas todos os posts daquele usuário receberam (ignora NULL).
- COUNT(DISTINCT p.id_post)
 - Conta quantos posts diferentes aquele usuário publicou.
- HAVING COUNT(c.id_post) > 3
 - Filtra para deixar só quem recebeu mais de 3 curtidas no total.
- ORDER BY total_curtidas DESC
 - Ordena do usuário mais curtido para o menos curtido (dentro dos que passaram no filtro).

Tabela base: Usuario

- Primeiro, o SQL “começa” pela cláusula FROM: FROM Usuario u
- Isso gera, conceitualmente, uma tabela com todos os usuários:

id_usuario	login
1	joao
2	maria
3	ana
4	lucas
5	empresa
6	sandra

7	paulo
8	rita
9	carlos
10	aline
11	breno
12	helen
13	pedro

14	flavia
15	igor

Primeiro JOIN: Usuario ⚡ Post

- LEFT JOIN Post p ON p.id_usuario = u.id_usuario
- LEFT JOIN: todos os usuários aparecem, mesmo se não tiverem posts.
- Para cada usuário, são ligados todos os posts que ele publicou.

A tabela Post (simplificada) é:

id_post	id_usuario	texto
1	1	Bem-vindos ao TUFCitter!
2	2	Primeiro post, animada para aprender!
3	3	Alguém aí curte IA?
4	2	Prova de BD chegando...
5	4	Testando o micro-blog novo!
...	...	...
29	9	Consegui um estágio novo!

Resultado parcial (exemplo com alguns usuários)

- Depois do LEFT JOIN, o resultado (tabela intermediária) fica algo assim:

u.id_usuario	u.login	p.id_post	p.id_usuario
1	joao	1	1
1	joao	7	1
2	maria	2	2
2	maria	4	2
2	maria	14	2
2	maria	16	2
..	...	..	..
15	igor	NULL	NULL

Segundo JOIN: ligar Post com Curtida

- LEFT JOIN Curtida c
ON c.id_post =
p.id_post
- A tabela Curtida é:
- Aqui não nos interessa quem curtiu, só quantas curtidas cada post recebeu, então só vamos usar c.id_post na contagem.

c.id_usuario (quem curtiu)	c.id_post (post curtido)
2	1
3	1
4	2
1	3
2	4
1	2
5	7
3	5
9	8
6	9
...	...

Como fica depois do JOIN com Curtida:

u.login	p.id_post	c.id_post (do JOIN)	interpretação
joao	1	1	post 1 curtido por alguém
joao	1	1	post 1 curtido por outra pessoa
joao	7	7	post 7 curtido por alguém
joao	15	NULL	post 15 sem curtidas
joao	24	24	post 24 curtido por alguém
joao	24	24	post 24 curtido por outra pessoa
joao	27	27	post 27 curtido por alguém
joao	27	27	post 27 curtido por outra pessoa

Resultado intermediário para um usuário (ex: joao)

- Posts de joao ($\text{id_usuario} = 1$): posts 1, 7, 15, 24, 27.
- Contando:
 - Post 1 $\rightarrow$ 2 curtidas
 - Post 7 $\rightarrow$ 1 curtaida
 - Post 15 $\rightarrow$ 0 curtidas
 - Post 24 $\rightarrow$ 2 curtidas
 - Post 27 $\rightarrow$ 2 curtidas
- Total de curtidas recebidas por joao $= 2 + 1 + 0 + 2 + 2 = 7$
- Total de posts de joao $= 5$ (1, 7, 15, 24, 27)
- A tabela “gigante” depois desse segundo LEFT JOIN contém uma linha para cada combinação:
 - usuário
 - post desse usuário
 - curtaida daquele post (se houver)
- Posts sem curtaida aparecem com $\text{c.id_post} = \text{NULL}$.

Agrupamento: GROUP BY

- GROUP BY u.id_usuario, u.login
- Isso significa:
 - Junta todas as linhas do mesmo usuário em um grupo.
 - Dentro de cada grupo, aplicamos as funções de agregação (COUNT, COUNT(DISTINCT ...)).
- Para cada usuário (grupo), calculamos:
 - COUNT(c.id_post) -- total de curtidas recebidas
 - COUNT(DISTINCT p.id_post) -- total de posts diferentes
- Lembre: COUNT(c.id_post) não conta NULL, então posts sem curtida não

Resultado do GROUP BY (todos os usuários)

login	total_curtidas	total_posts
maria	9	5
joao	7	5
lucas	6	4
sandra	5	3
carlos	5	3
ana	4	3
flavia	4	2
empresa	2	2
breno	3	2
paulo	0	0
rita	0	0
aline	0	0
helenia	0	0
pedro	0	0
igor	0	0

Filtro de grupos: **HAVING COUNT(c.id_post) > 3**

- HAVING COUNT(c.id_post) > 3
- é um filtro sobre o resultado do GROUP BY (ou seja, sobre os grupos).
- Só ficam os usuários cujo total_curtidas > 3.

Filtro de grupos: **HAVING COUNT(c.id_post) > 3**

- Aplicando o filtro na tabela anterior, sobram:

login	total_curtidas	total_posts
maria	9	5
joao	7	5
lucas	6	4
sandra	5	3
carlos	5	3
ana	4	3
flavia	4	2

Ordenação final: ORDER BY total_curtidas DESC

- ORDER BY total_curtidas DESC;
- Ordena do maior para o menor número de curtidas totais.
- A ordem já está assim na tabela acima, mas formalmente o resultado final é:

Resultado

login	total_curtidas	total_posts
maria	9	5
joao	7	5
lucas	6	4
sandra	5	3
carlos	5	3
ana	4	3
flavia	4	2

02. Usuários sem Perfil ou sem Endereço

- Técnica: LEFT JOIN, WHERE com IS NULL
- Considere a tabela de usuários, perfis e endereços. Liste o login de todos os usuários que não possuem perfil cadastrado ou não possuem endereço cadastrado. Não é necessário identificar o motivo (basta listar o login). Usuários que têm tanto perfil quanto endereço não devem aparecer no resultado.

Resposta

```
SELECT
    u.login
FROM Usuario u
LEFT JOIN Perfil p
    ON p.id_usuario = u.id_usuario
LEFT JOIN EnderecoUsuario e
    ON e.id_usuario = u.id_usuario
WHERE p.id_usuario IS NULL
    OR e.id_usuario IS NULL;
```

03. Usuários e Seus Seguidores

- Técnica: LEFT JOIN duplo, COUNT, GROUP BY, ORDER BY
- Considere a tabela de usuários e as relações de seguidores. Para cada usuário cadastrado, apresente: O login do usuário, A data em que o usuário foi cadastrado no sistema, O total de seguidores que esse usuário possui (ou seja, a quantidade de outros usuários que o seguem), O total de pessoas que esse usuário segue (ou seja, a quantidade de outros usuários que ele está seguindo). Mostre todos os usuários do sistema, incluindo aqueles que não possuem seguidores ou que não seguem ninguém (nesses casos, mostre o total como zero). Ordene o resultado do usuário com maior número de seguidores para o de menor.

Resposta

```
SELECT
    u.login,
    u.data_cadastro,
    COUNT(seguidores.id_seguidor) AS total_seguidores,
    COUNT(seguinto.id_seguinto) AS total_seguinto
FROM Usuario u
LEFT JOIN Seguir AS seguidores
    ON seguidores.id_seguinto = u.id_usuario    -- quem segue o usuário
LEFT JOIN Seguir AS seguinto
    ON seguinto.id_seguidor = u.id_usuario    -- quem o usuário segue
GROUP BY
    u.id_usuario, u.login, u.data_cadastro
ORDER BY
    total_seguidores DESC,
    u.login;
```


04. Posts Sem Curtidas

- Técnica: LEFT JOIN, WHERE IS NULL
- Considere todos os posts publicados. Liste, para cada post que não recebeu nenhuma curtida: O login do usuário que publicou. O texto do post. A data e hora da publicação.

Resposta

```
SELECT
    u.login,
    p.texto,
    p.data_hora
FROM Post p
JOIN Usuario u
    ON u.id_usuario = p.id_usuario
LEFT JOIN Curtida c
    ON c.id_post = p.id_post
WHERE c.id_post IS NULL;
```

05. Telefones de Usuários que Publicaram Posts com Curtidas

- Técnica: INNER JOIN múltiplo, DISTINCT
- Considere todos os usuários que já publicaram pelo menos um post que tenha recebido curtidas. Liste, para cada um desses usuários: O login. Todos os telefones cadastrados para esse usuário (sem repetições, mesmo se houver mais de um post com curtidas por esse usuário). Cada linha deve mostrar um login e um telefone. Se o usuário tiver mais de um telefone, aparecerá uma linha para cada telefone.

Resposta

```
SELECT DISTINCT
    u.login,
    t.telefone
FROM Usuario u
JOIN Post p
    ON p.id_usuario = u.id_usuario
JOIN Curtida c
    ON c.id_post = p.id_post
JOIN UsuarioTelefone t
    ON t.id_usuario = u.id_usuario
ORDER BY u.login, t.telefone;
```

06. Usuários que Não Enviaram Nem Receberam Mensagens

- Técnica: Subconsulta com NOT IN
- A partir das mensagens privadas do sistema, liste todos os usuários que nunca enviaram nem receberam nenhuma mensagem privada, ou seja, que não aparecem como remetente nem como destinatário em nenhuma mensagem. Mostre apenas o login de cada usuário no resultado.

Resposta

```
SELECT
    u.login
FROM Usuario u
WHERE u.id_usuario NOT IN (
    SELECT id_remetente FROM MensagemPrivada
    UNION
    SELECT id_destinatario FROM MensagemPrivada
);
```

07. Post com Mais Curtidas de Cada Usuário

- Técnica: JOIN, LEFT JOIN, GROUP BY, COUNT, ORDER BY
- Para todos os posts publicados no sistema, mostre: O login do usuário autor do post. O texto do post. A quantidade de curtidas recebidas por esse post. Apresente o resultado ordenado do post com mais curtidas para o com menos curtidas.

Resposta

```
SELECT
    u.login,
    p.texto,
    COUNT(c.id_usuario) AS total_curtidas
FROM Post p
JOIN Usuario u
    ON u.id_usuario = p.id_usuario
LEFT JOIN Curtida c
    ON c.id_post = p.id_post
GROUP BY u.login, p.id_post, p.texto
ORDER BY total_curtidas DESC, p.data_hora;
```


08. Usuários que Mais Curtiram Posts

- Técnica: JOIN, COUNT, GROUP BY, ORDER BY
- Liste os usuários que mais realizaram curtidas em posts no sistema. Se houver empate na quantidade de curtidas, inclua todos os usuários empatados nessa posição. Para cada um, mostre o login e a quantidade total de curtidas que ele fez.

Resposta

```
SELECT
    u.login,
    COUNT(*) AS total_curtidas
FROM Curtida c
JOIN Usuario u
    ON u.id_usuario = c.id_usuario
GROUP BY
    u.id_usuario, u.login
HAVING
    COUNT(*) = (
        SELECT MAX(sub.qtd)
        FROM (
            SELECT
                COUNT(*) AS qtd
            FROM Curtida
            GROUP BY id_usuario
        ) AS sub
    );
```

09. Consulta com Todos os Usuários, Posts e Curtidas

- Técnica: JOIN, ORDER BY
- Para cada usuário que publicou pelo menos um post, mostre: O login do usuário. O texto de cada post publicado por ele. A data e hora em que o post foi curtido (se houver curtida naquele post). Inclua todos os posts, mesmo que não tenham recebido curtidas. Se o post não tiver curtidas, o campo data/hora deve aparecer vazio/nulo.

Resposta

```
SELECT
    u.login,
    p.texto,
    c.data_hora AS data_hora_curtida
FROM Usuario u
JOIN Post p
    ON p.id_usuario = u.id_usuario
LEFT JOIN Curtida c
    ON c.id_post = p.id_post
ORDER BY u.login, p.data_hora, c.data_hora;
```

10. Total de posts por usuário

- Técnica: LEFT JOIN, COUNT, GROUP BY, ORDER BY
- Para cada usuário, mostre o login e a quantidade de posts que publicou. Apresente os resultados em ordem decrescente de quantidade de posts.

Resposta

```
SELECT
    u.login,
    COUNT(p.id_post) AS total_posts
FROM Usuario u
LEFT JOIN Post p
    ON p.id_usuario = u.id_usuario
GROUP BY u.login
ORDER BY total_posts DESC, u.login;
```

Bibliografia Básica

- ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 6 ed. Pearson/Addison-Wesley, 2011. ISBN: 9788579360855
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. Rio de Janeiro: Elsevier: Campus, 2012. ISBN 9788535245356
- HEUSER, C. A. Projeto de banco de dados. 6 ed. Bookman, 2009. ISBN: 9788577803828



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

